

グラフ作成1

医療データ科学実習

(Practice of Biomedical Data Science)

第7回 2026年6月2日

医療データ科学

matsui.kota.8k@kyoto-u.ac.jp

臨床統計学

emoto.ryo.5m@kyoto-u.ac.jp

Slidoで質疑応答に参加しよう

医療データ科学実習第2回

医療データ科学実習第2回
2025/04/15~2025/04/22
#2974 065

ライブインタラクシ...

Slido を切り替え

ダークモード ☐

Slido について

Slido を無料で試す

Q&A

Polls

質問を入力

人気 最近

質問投稿部分

1 個の質問

匿名
6 日前

0

人の質問に投票できる
人気の質問は上位に表示される

テスト質問です

Moderator
6 日前

テスト質問了解です

- 匿名で質疑応答に参加できるプラットフォーム
- スマホからでも簡単に利用できます
- 講義後1週間解放しておくので自由に質問やコメントを投稿してください

slido

参加

URL: <https://app.sli.do/event/fYcDWF8dgb9LyKcnTnZpxN>



提出課題

本日、第7回の実習で実行したRscriptファイルを提出してください

提出方法: KULMSの課題提出機能

提出期限: 6/3(水) 23:55

Chap1. Rパッケージを用いた グラフィックス

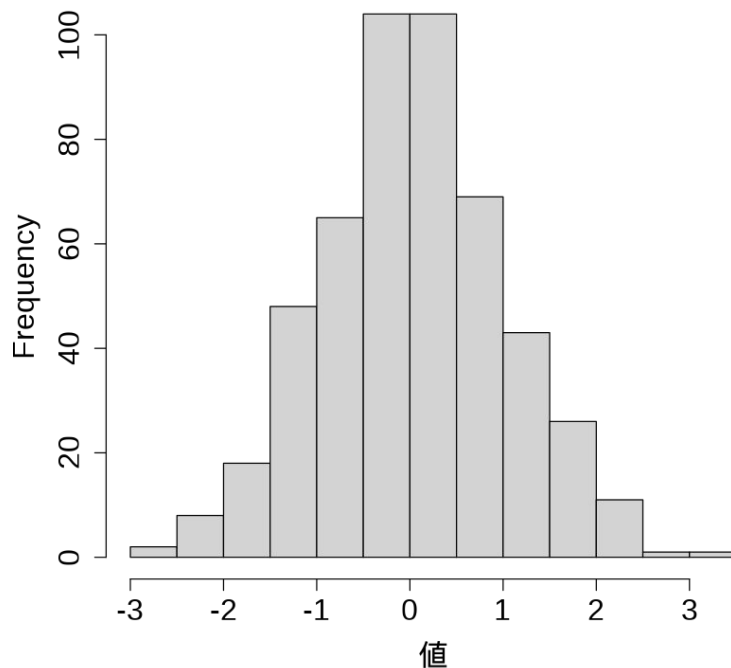
- 1次元の連続変数の経験分布
(ヒストグラム, 経験累積分布)
- 箱ひげ図
- 複数のプロットの重ね描き

ggplot2 によるデータのヒストグラム作成

ヒストグラム (histogram)

- 連続的な数値データの分布(頻度)を視覚的に表現する棒グラフの一種
- データがどの範囲に多く存在するか(分布の形)を把握するためによく使われる
 - 棒の幅=ビンの幅(通常は等間隔)
 - 棒の高さ=ビンに含まれるデータの数

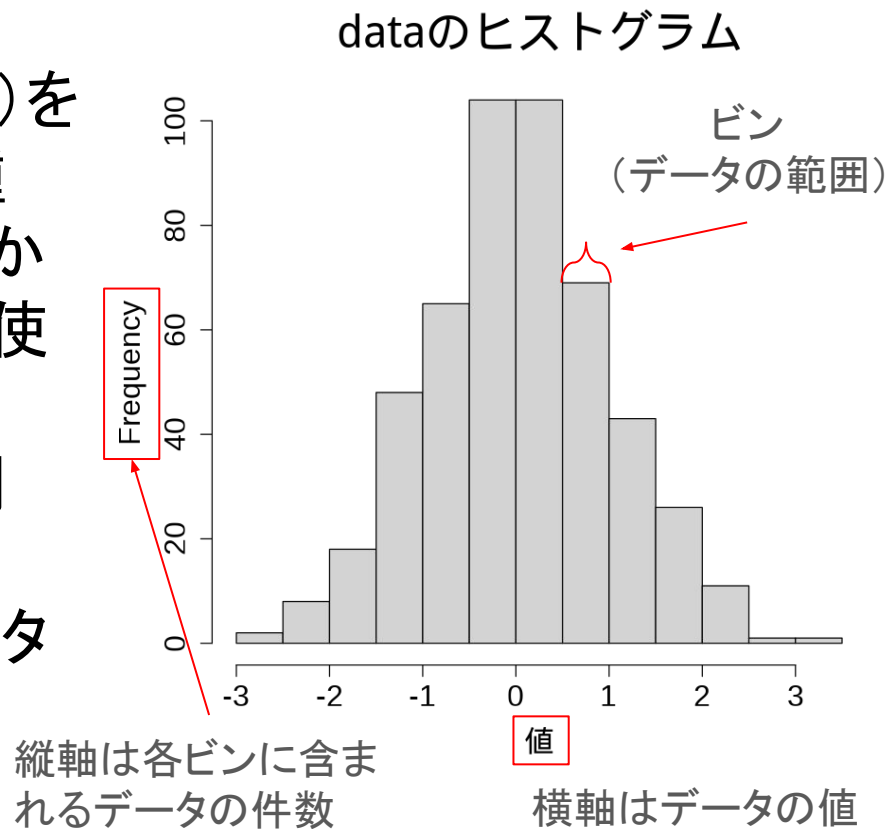
dataのヒストグラム



ggplot2 によるデータのヒストグラム作成

ヒストグラム (histogram)

- 連続的な数値データの分布 (頻度) を視覚的に表現する棒グラフの一種
- データがどの範囲に多く存在するか (分布の形) を把握するためによく使われる
 - 棒の幅=ビンの幅 (通常は等間隔)
 - 棒の高さ=ビンに含まれるデータの数



ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
# サンプルデータの生成
set.seed(123)
data <- data.frame(x = rnorm(500, mean = 0, sd = 1))

# バイン数を指定してヒストグラムを頻度で描く
ggplot(data, aes(x = x)) +
  geom_histogram(aes(y = ..count..), bins = 10, fill = "grey", color = "black") +
  labs(title = "データのヒストグラム", x = "値", y = "頻度") +
  theme_bw(base_size = 14) +
  theme(
    plot.title       = element_text(size = 24, face = "bold", hjust = 0.5), # タイトル
    plot.subtitle    = element_text(size = 20, hjust = 0.5),                # サブタイトル
    axis.title.x     = element_text(size = 20),                            # x 軸タイトル
    axis.title.y     = element_text(size = 20),                            # y 軸タイトル
    axis.text        = element_text(size = 20)                             # 軸目盛り
  )
```

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
# サンプルデータの生成
set.seed(123)
data <- data.frame(x = rnorm(500, mean = 0, sd = 1))
```


ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
# サンプルデータの生成
```

```
set.seed(123) —→ 乱数シードを固定
```

```
data <- data.frame(x = rnorm(500, mean = 0, sd = 1))
```

→ 標準正規分布からの乱数を 500 個生成
mean(平均パラメータ)と sd(標準偏差パラメータ)を変更
することで、一般の正規分布からの乱数生成もできる

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

ビン数を指定してヒストグラムを頻度で描く

```
ggplot(data, aes(x = x)) +  
  geom_histogram(aes(y = ..count..), bins = 10, fill = "grey", color = "black") +  
  labs(title = "データのヒストグラム", x = "値", y = "頻度") +  
  theme_bw(base_size = 14) +  
  theme(  
    plot.title      = element_text(size = 24, face = "bold", hjust = 0.5), # タイトル  
    plot.subtitle   = element_text(size = 20, hjust = 0.5),                # サブタイトル  
    axis.title.x    = element_text(size = 20),                            # x 軸タイトル  
    axis.title.y    = element_text(size = 20),                            # y 軸タイトル  
    axis.text       = element_text(size = 20)                             # 軸目盛り  
  )
```

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
ggplot(data, aes(x = x))
```

+

→ 次の処理への接続演算子

→ プロットの基本レイヤを作成

- 第1引数は data (データフレーム)
- 第2引数はデータフレーム内のどの変数を何のために使うかを指定する関数
→ 上の例ではプロットの x 軸にデータの列 x を対応付けている

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
geom_histogram(aes(y = ..count..), bins = 10, fill = "grey", color = "black") +
```

- 基本レイヤの次には `geom_xxx (geometry)` という名前の関数を加える例:
 - ヒストグラム : `geom_histogram()`
 - 散布図 : `geom_point()`使う `geom` 関数の種類によって、中の `aes()` で指定するものが変わる
- ヒストグラムの場合
 - `aes(y=..count..)` 縦軸を（絶対）頻度でプロットする
 - `aes(y=..density..)` とすると縦軸が相対頻度になる
 - `bins=10` でビン数（区間数）を10に固定（**base関数とは異なる**）
 - `fill="grey"` で棒グラフの内部色を指定
 - `color="black"` で棒グラフの枠線色を指定

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
labs(title = "データのヒストグラム", x = "値", y = "頻度") +
```

→ タイトルと軸ラベルを設定するレイヤ

- title="データのヒストグラム" : プロットのタイトル
- x="値" : x軸の変数名
- y="頻度" : y軸の変数名

ggplot2 によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

`theme_bw(base_size = 14)` → プロットの全てのテキストの文字サイズを一括変更

```
theme(  
  plot.title       = element_text(size = 24, face = "bold", hjust = 0.5), # タイトル  
  plot.subtitle    = element_text(size = 20, hjust = 0.5),                # サブタイトル  
  axis.title.x     = element_text(size = 20),                            # x 軸タイトル  
  axis.title.y     = element_text(size = 20),                            # y 軸タイトル  
  axis.text        = element_text(size = 20)                             # 軸目盛り  
)
```

→ 特定の要素の文字サイズを変更

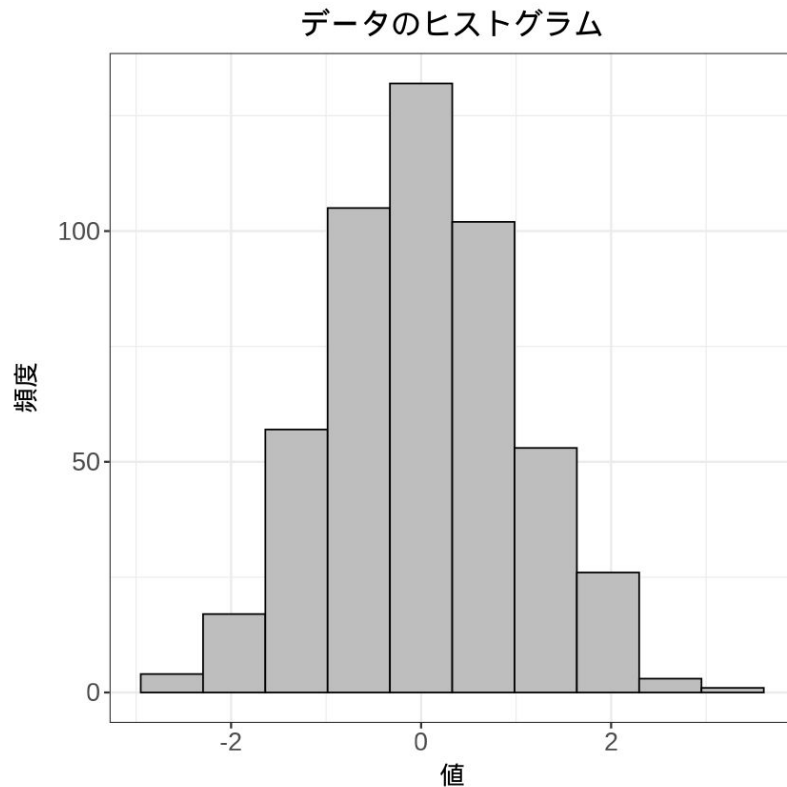
- `face="bold"` : 文字をbold体にする
- `hjust=0.5` : ラベルの左右位置調整
(上下調整の場合は `vjust=0.5` など)

ggplot2 によるデータのヒストグラム作成

サンプルコードの実行結果

演習: RStudioでサンプルコードを実行してヒストグラムを描画してみよう

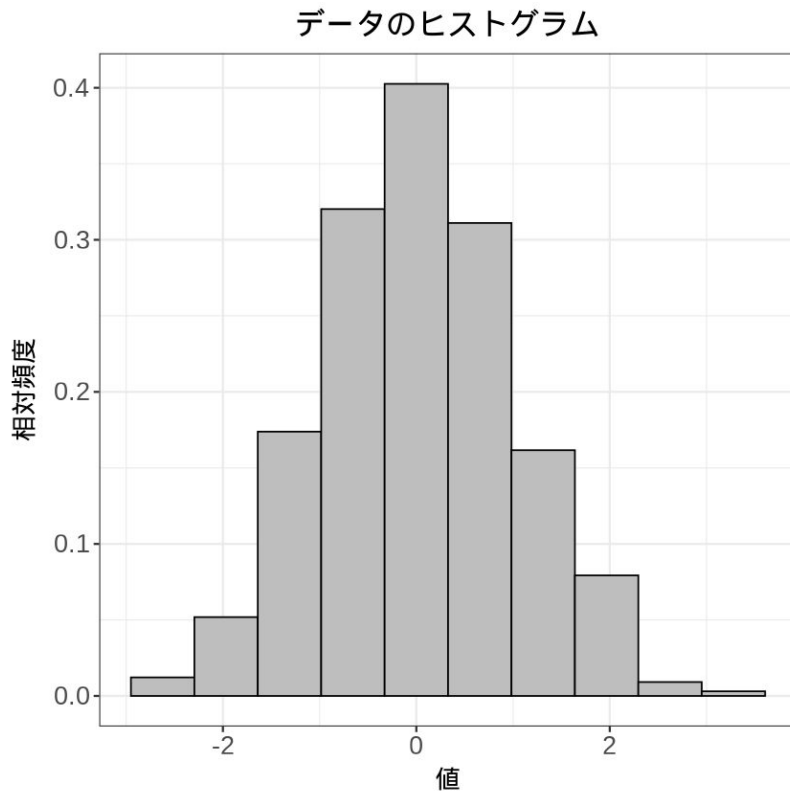
演習: `count` と `density` オプションを切り替えてヒストグラムを描画してみよう



count での描画

ggplot2 によるデータのヒストグラム作成

density オプションによる相対頻度の描画



density での描画

ggplot2 によるデータの経験累積分布関数の可視化

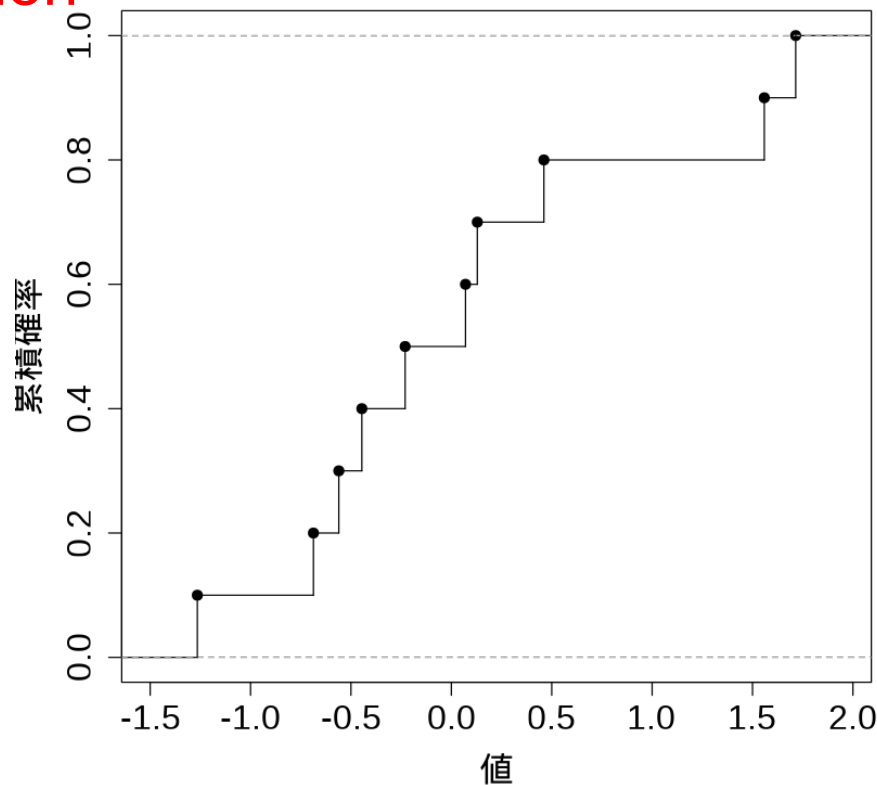
累積分布関数(cumulative distribution function, cdf)

確率変数 X の累積分布関数 $F(X)$ は、ある実現値 x に対して

$$F(x) = \Pr(X \leq x)$$

で定義される

データのECDF



ggplot2 によるデータの経験累積分布関数の可視化

累積分布関数(cumulative distribution function, cdf)

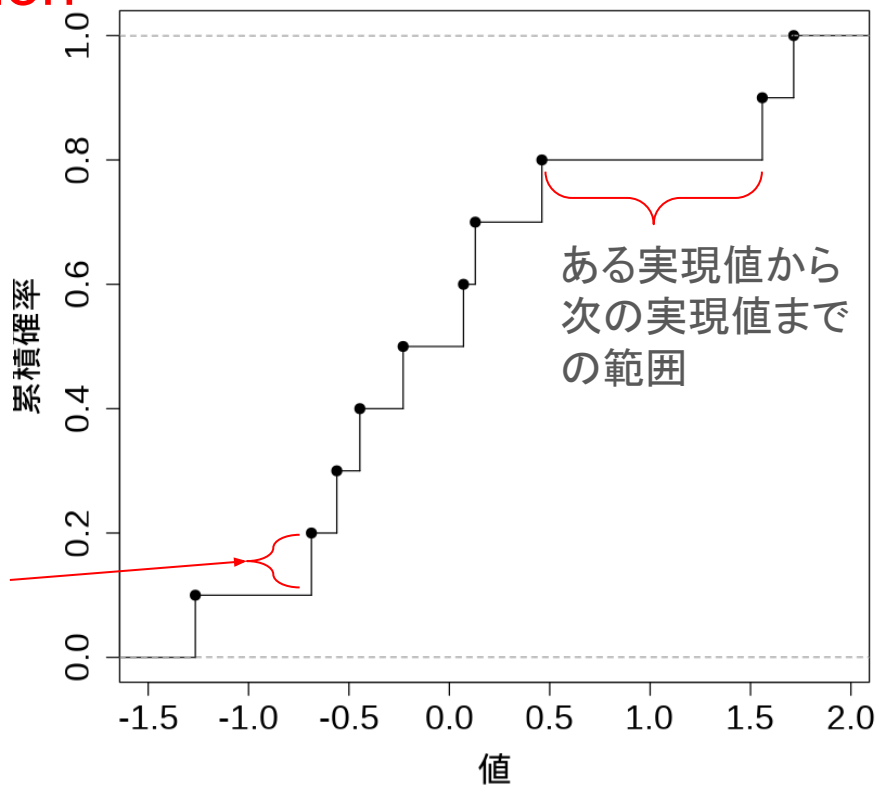
確率変数 X の累積分布関数 $F(X)$ は、ある実現値 x に対して

$$F(x) = \Pr(X \leq x)$$

で定義される

観測値が現れたとき、その点より小さいか等しい値のデータの割合(累積確率)が増える → 結果として、cdfの値(縦軸)が「ジャンプ」する

データのECDF



ggplot2 によるデータの経験累積分布関数の可視化

cdf描画のサンプルコード

```
set.seed(123)
data <- data.frame(x = rnorm(50, mean = 0, sd = 1))

ggplot(data, aes(x = x)) +
  stat_ecdf(geom = "step", color = "blue") +
  labs(title = "累積頻度分布 (ECDF)", x = "x", y = "累積確率") +
  theme(
    plot.title = element_text(size = 24, face = "bold"),
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 16)
  )
```

ggplot2 によるデータの経験累積分布関数の可視化

cdf描画のサンプルコード

```
stat_ecdf(geom = "step", color = "blue") +
```

ggplot2 によるデータの経験累積分布関数の可視化

cdf描画のサンプルコード

```
stat_ecdf(geom = "step", color = "blue") +
```

→ ggplot2 の統計関数の一つで、データに基づいた
経験累積分布関数（ECDF）を描画

→ 階段状の線（ステップ関数）としてECDFを描く

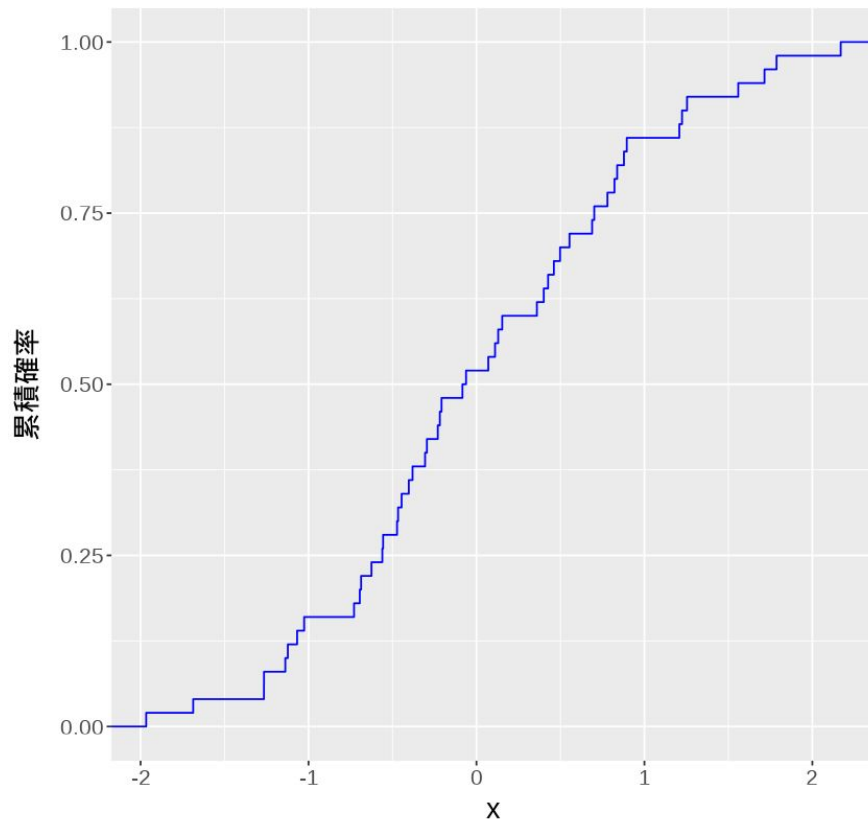
ggplot2 によるデータの経験累積分布関数の可視化

cdf描画の°プロット結果

演習 : RStudioでコードを実行して標準正規分布の `ecdf` をプロットしてみよう

*ベース関数と違い, 最初からジャンプを表す縦線が入っている

累積頻度分布 (ECDF)

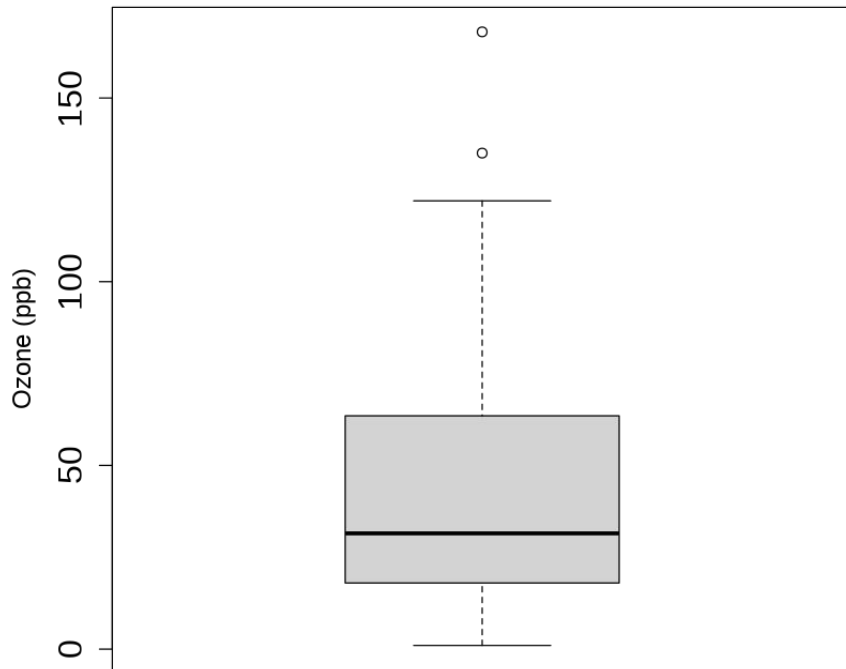


boxplot によるデータの箱ひげ図の描画

箱ひげ図 (box plot)

- データの分布・ばらつき・外れ値を視覚的に要約するためのグラフ
- **5つの要約統計量**をもとに構成され、データの中心・広がり・非対称性・外れ値などを直感的に把握できる

Ozone濃度のボックスプロット

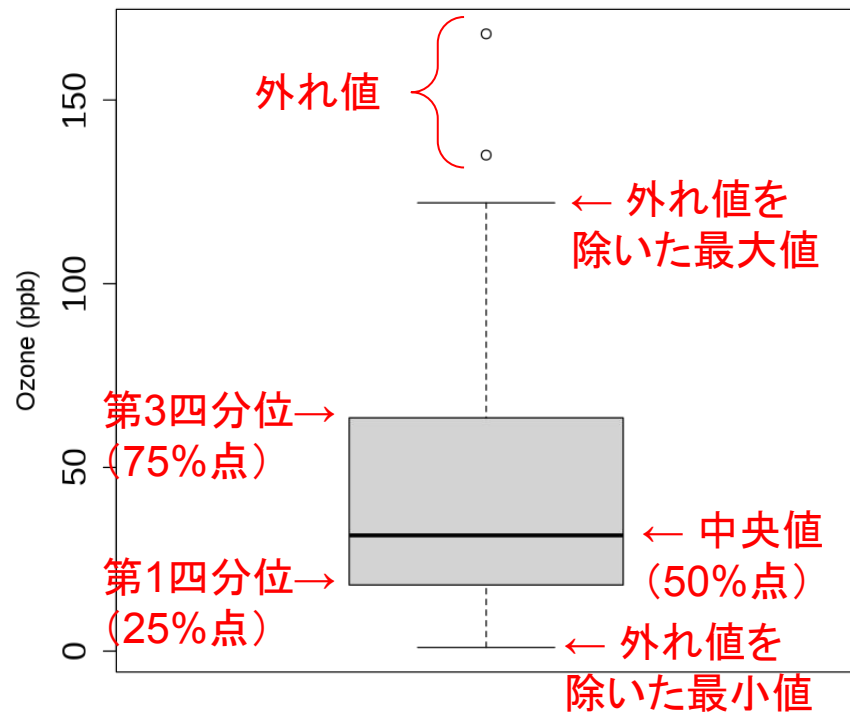


boxplot によるデータの箱ひげ図の描画

箱ひげ図 (box plot)

- データの分布・ばらつき・外れ値を視覚的に要約するためのグラフ
- **5つの要約統計量**をもとに構成され、データの中心・広がり・非対称性・外れ値などを直感的に把握できる

Ozone濃度のボックスプロット



boxplot によるデータの箱ひげ図の描画

箱ひげ図描画のサンプルコード

```
ozone_data <- na.omit(data.frame(Ozone = airquality$Ozone))
```

箱ひげ図を描く

```
ggplot(ozone_data, aes(x = "", y = Ozone)) +  
  geom_boxplot(fill = "skyblue", color = "black") +  
  labs(title = "Ozone の箱ひげ図", y = "Ozone", x = "") +  
  theme_minimal(base_size = 20)
```

boxplot によるデータの箱ひげ図の描画

箱ひげ図描画のサンプルコード

基本レイヤ

```
ggplot(ozone_data, aes(x = "", y = Ozone)) +  
  geom_boxplot(fill = "skyblue", color = "black") +
```

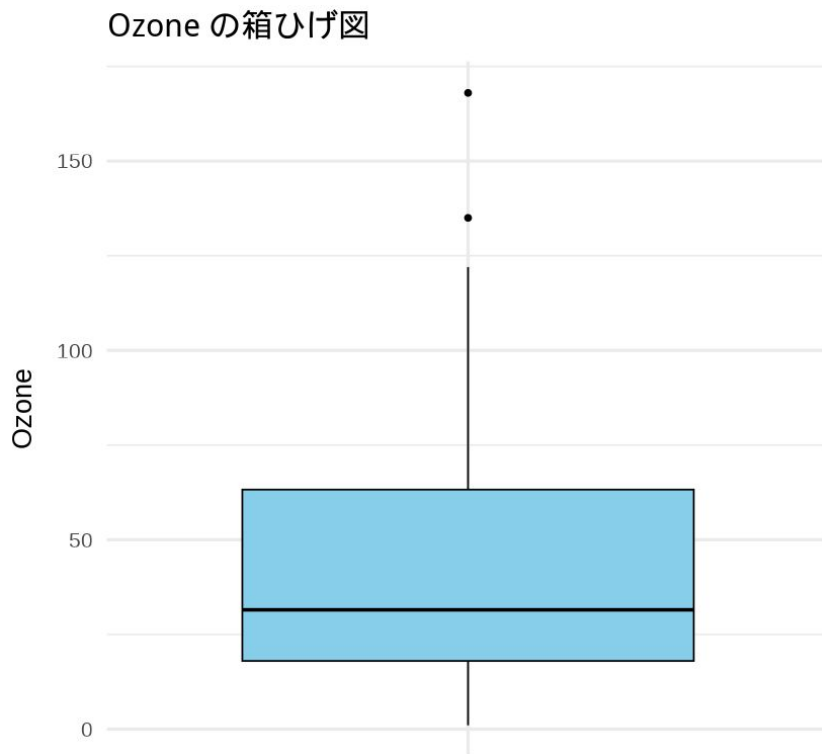
geom_boxplot() 関数で箱ひげ図を描画

boxplot によるデータの箱ひげ図の描画

箱ひげ図描画の実行結果

演習 : RStudioでコードを実行して箱ひげ図をプロットしてみよう

演習 : 欠損値の処理を入れた場合と入れない場合でプロットが変わるかどうかを確認しよう



複数のプロットを重ねて表示する

The Old Faithful Geyser(間欠泉)データ

- Rの組み込みデータセット

```
print(faithful)
```

	eruptions	waiting
1	3.600	79
2	1.800	54
3	3.333	74
4	2.283	62
5	4.533	85
6	2.883	55
7	4.700	88
8	3.600	85
9	1.950	51



1列目: 噴出の継続時間, 2列目: 間欠泉の噴出の間隔(いずれも分)

複数のプロットを重ねて表示する

ggplot2では、基本的にレイヤを重ねるだけでプロットの重ね表示が可能

プロットの重ね合わせのサンプルコード

data <- faithful ヒストグラムと密度関数を重ねてプロットする

```
ggplot(data, aes(x = eruptions)) +  
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue",  
    color = "black", alpha = 0.6) +  
  geom_density(color = "red", size = 1.2) +  
  labs(title = "ヒストグラムと密度曲線の重ね描き",  
    x = "噴出時間", y = "密度") +  
  theme_minimal(base_size = 20)
```

複数のプロットを重ねて表示する

プロットの重ね合わせのサンプルコード

```
ggplot(data, aes(x = eruptions)) +  
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue",  
  color = "black", alpha = 0.6) +  
  geom_density(color = "red", size = 1.2) +
```

基本レイヤ

ヒストグラムレイヤ

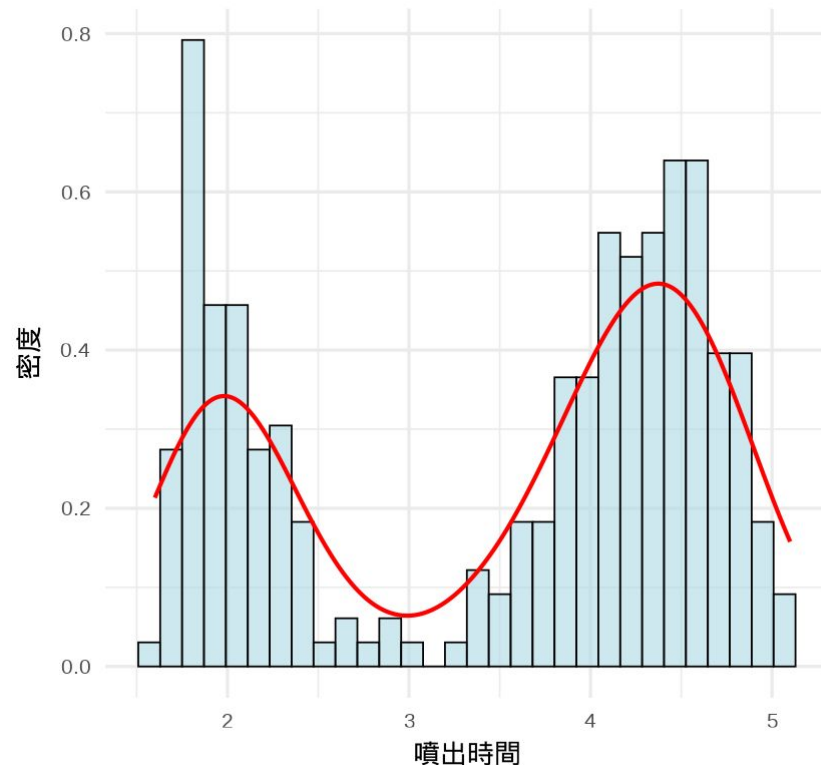
密度関数レイヤ

複数のプロットを重ねて表示する

プロットの重ね合わせの実行結果

演習 : RStudioでコードを実行してプロットを重ねて表示してみよう

ヒストグラムと密度曲線の重ね描き



Chap 2. 散布図

- 2つの変数の関係の可視化
- グラフの体裁の指定

散布図: 二つの連続変数の関係の視覚化

二つの連続変数の関係を点で示す最も基本的なグラフ

- 散布図のそれぞれの点は、対応する二つの連続変数の値の組み合わせを表している
- ggplot2 では data.frame を入力し、列名を指定することで描画
- 散布図は ggplot() + geom_point() で描画

ggplot2での基本操作:

```
ggplot(data = "データフレーム名",  
       mapping = aes(x = "x軸の列名", y = "y軸の列名")) +  
  geom_point()
```

散布図:コード例

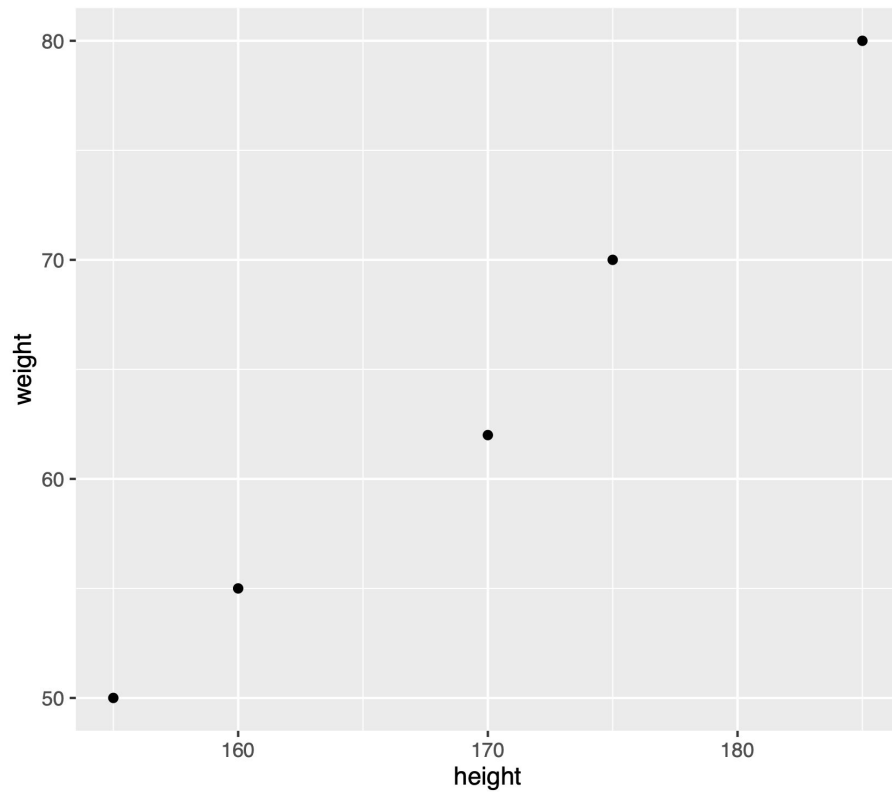
あるグループの人々の身長と体重の散布図の作成を想定

```
install.packages("ggplot2")
library(ggplot2)

# データ準備
data <- data.frame(
  height = c(155, 170, 175, 160, 185),
  weight = c(50, 62, 70, 55, 80)
)
print(data)

# ggplot2 で散布図
ggplot(data=data, aes(x = height, y = weight)) +
  geom_point()
```

散布図：出力例

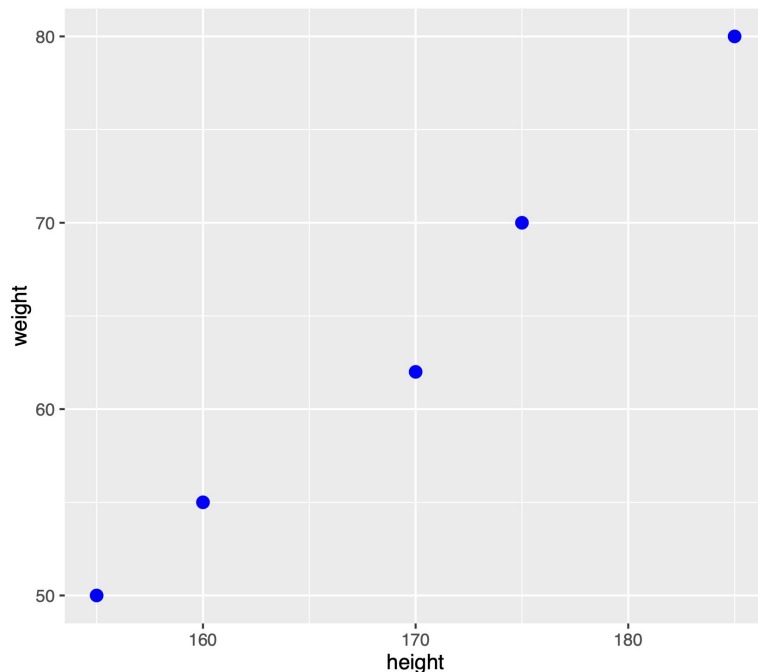


散布図: 点の設定

散布図の点の色や形状、大きさは `geom_point()` の下記の引数で設定

- `color` : 点の色を指定
- `shape` : 点の形状を指定
- `size` : 点の大きさを指定

```
ggplot(data=data,  
  aes(x = height,  
      y = weight)) +  
  geom_point(  
    color = "blue",  
    size = 3,  
    shape = 16)
```



色の指定

- `color=数値`による指定と色の対応

1	2	3	4	5	6	7	8
							

- 文字列による指定: `colors()`関数で一覧を取得できる
- 16進数カラーコードでも指定できる(`col="#DF536B"`)

点の形状の指定

- `shape=数値`による指定と形状の対応

0	1	2	3	4	5	6	7	8
								
9	10	11	12	13	14	15	16	17
								
18	19	20	21	22	23	24	25	
								

- 21-25 は塗りつぶしが可能な形状
- `shape` に文字を指定するとその文字がプロットされる(`shape="A"`)

散布図: タイトルとラベル

適切なタイトルと軸ラベルは図の正確な解釈に不可欠

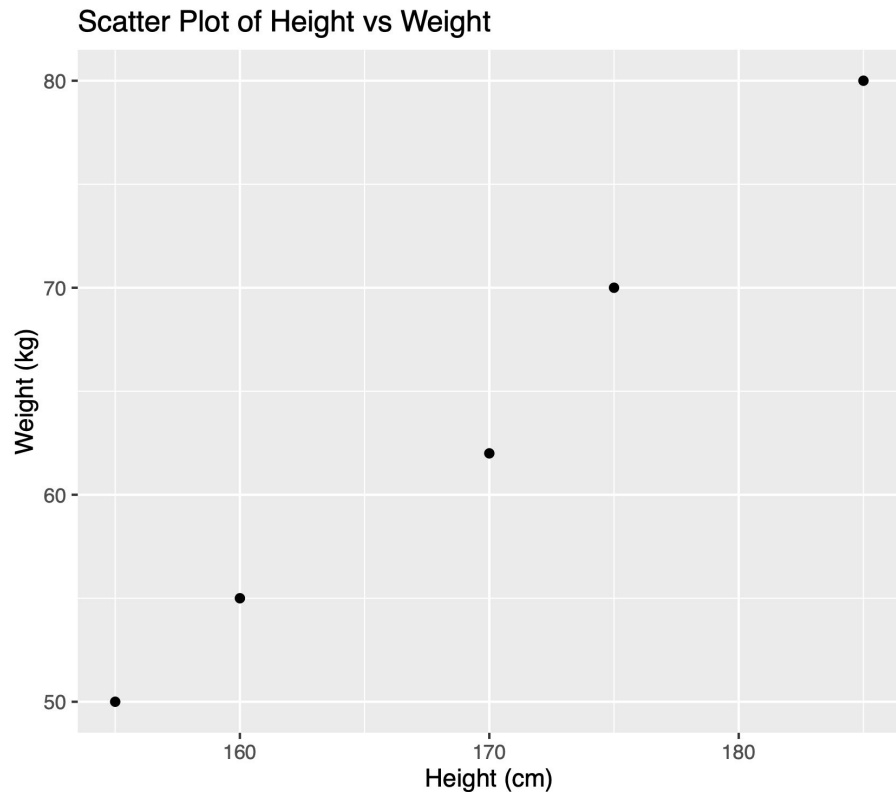
- タイトルや軸ラベルは `ggplot() + labs()` で追加
- `labs()` はタイトルや軸ラベルの"内容"を指定する関数
- タイトルとラベルの大きさは指定できない(後述)

`labs()` を用いた設定の仕方:

```
labs(title = "タイトルの文字列",  
      x = "x軸ラベルの文字列",  
      y = "y軸ラベルの文字列")
```

散布図: タイトルとラベル -例-

```
ggplot(data, aes(x = height,  
                  y = weight)) +  
  geom_point() +  
  labs(  
    title = "Scatter Plot of  
Height vs Weight",  
    x = "Height (cm)",  
    y = "Weight (kg)")
```



散布図: 複数のグループの散布図

複数のグループを色や形で区別して表示するには

`aes()` 内の引数 `color`, `shape` にグループを表す変数名を指定

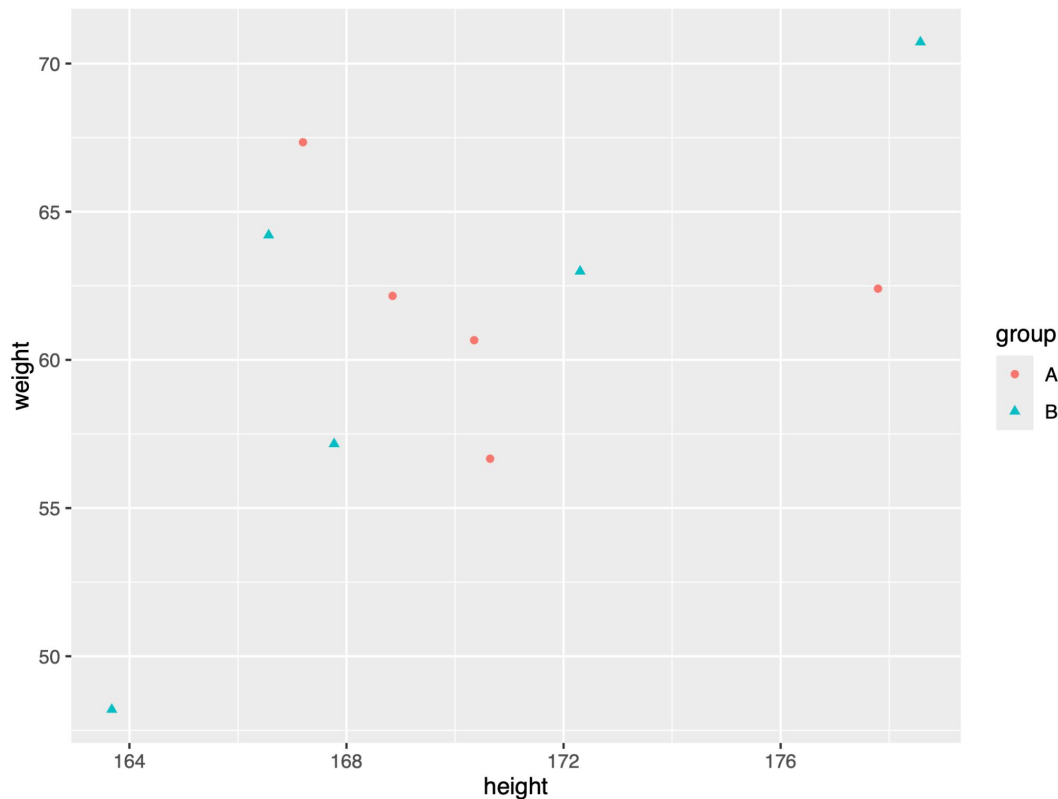
- グループごとに色や形が自動で割り当てられる
- 自動で描画範囲が定まる(まとめて描くので事前設定は必要なし)
- 自動で凡例が生成される

```
ggplot(data = "データフレーム名",  
       mapping = aes(x = "x軸の列名", y = "y軸の列名",  
                     color = "グループを表す変数名",  
                     shape = "グループを表す変数名")) +  
geom_point()
```

散布図: 複数のグループの散布図 -コード例-

```
# サンプルデータ作成
set.seed(123)
data2 <- data.frame(
  height = rnorm(10, mean = 170, sd = 5),
  weight = rnorm(10, mean = 60, sd = 6),
  group  = rep(c("A", "B"), each = 5))
# データ確認
print(data2)
# color, shapeを指定して描画
ggplot(data2, aes(x = height, y = weight,
  color = group, shape = group)) +
  geom_point()
```

散布図：応用 – 複数のグループの散布図例 –



散布図: 図のテーマの指定

`theme()` 関数: ggplot2 の描画全体の体裁を調整

- 例えば `labs()` で軸ラベルの内容を指定するのに対し `theme()` では軸ラベルの見た目を調整
- 各要素の文字サイズ、色、位置を細かく設定可能
- 凡例の配置、背景などの全体的な要素も設定可能

散布図: theme() によるタイトル、軸ラベルの設定

以下の theme() 関数の引数を設定

- plot.title: タイトルのスタイルの設定
- axis.title: 軸ラベルのスタイルの設定
- axis.text: 軸目盛りラベルのスタイルの設定

引数に element_text() 関数を使用して以下の指定ができる

- size: 対応するtextの文字サイズ(単位: pt)
- hjust: 対応するtextの左詰(0), 中央揃え(0.5), 右詰(1)

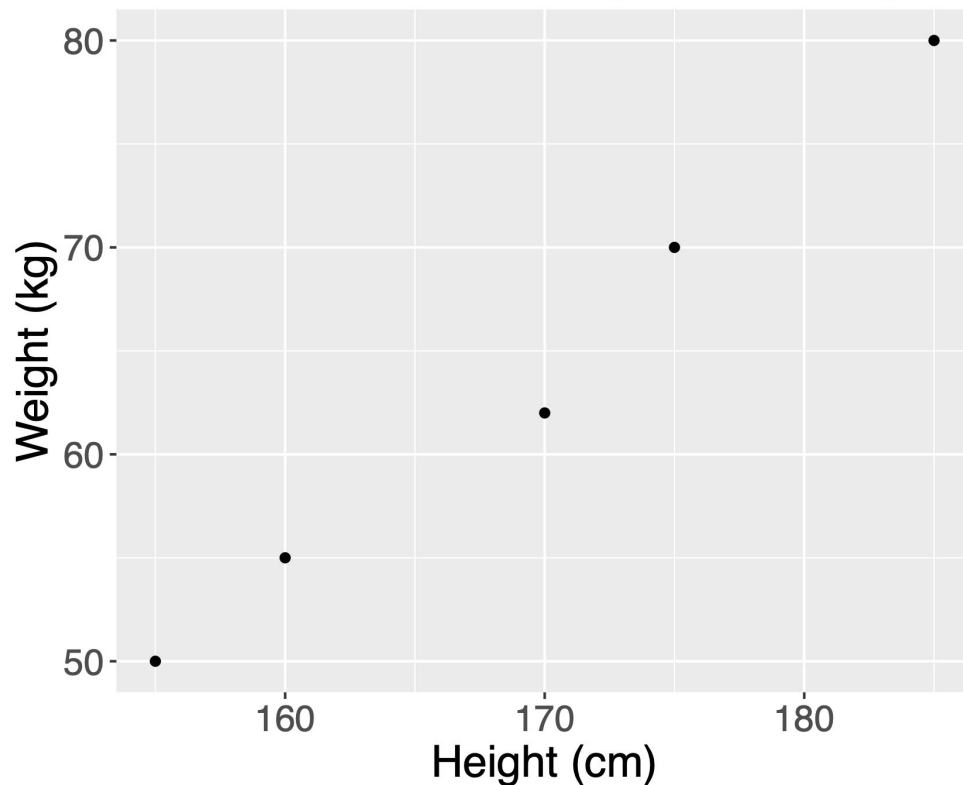
```
theme(plot.title = element_text(size = 文字サイズ))
```

散布図: タイトル、軸ラベルの設定 -コード例-

```
ggplot(data, aes(x = height, y = weight)) +  
  geom_point() +  
  labs(title = "Scatter Plot of Height vs Weight",  
        x = "Height (cm)", y = "Weight (kg)") +  
  theme(  
    plot.title = element_text(size = 24, hjust=0.5), #24pt, 中央揃え  
    axis.title = element_text(size = 18), # 軸ラベルは18pt  
    axis.text = element_text(size = 15)) # 軸メモリラベルは15pt
```

散布図: タイトル、軸ラベルの設定 -出力例-

Scatter Plot of Height vs Weight



散布図: テーマ全体の変更

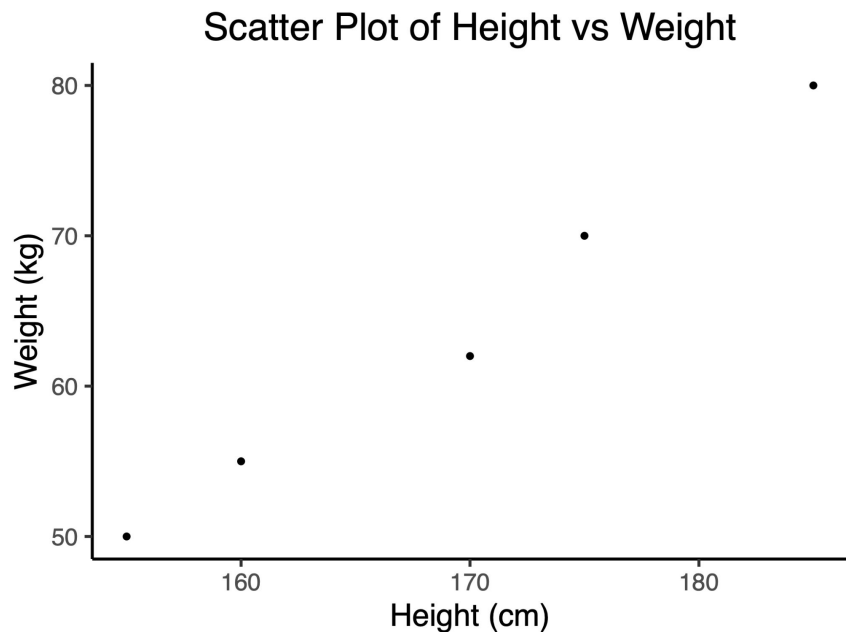
予め用意されたテーマが使用できる

- 描画スタイルの初期設定を変更したい場合に有用
- theme() と組み合わせて調整可能
 - theme()を組み込みテーマの後に書く必要がある
- theme_classic(), theme_minimal() などを使用することが多い
- theme_classic() の引数 base_size で全体的に文字のサイズを指定

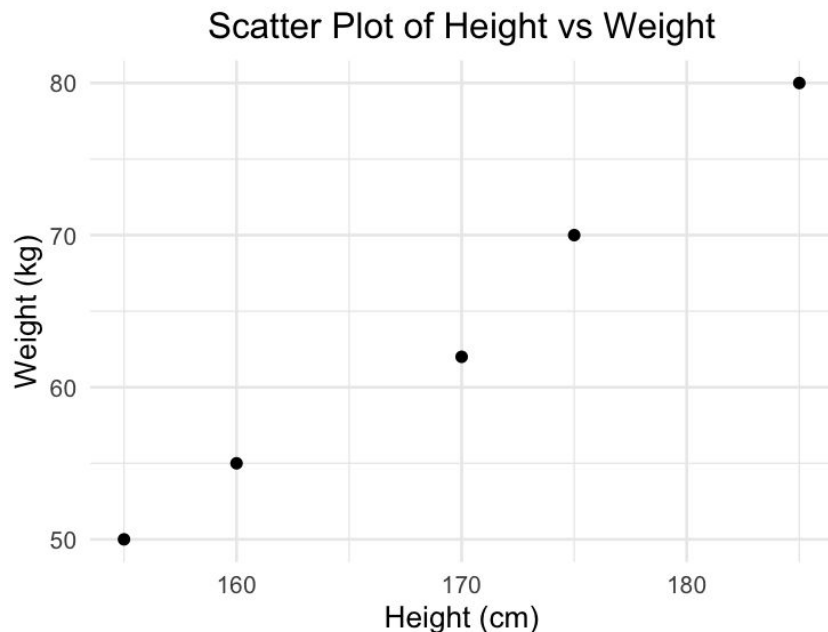
```
ggplot(data, aes(x = height, y = weight)) +  
  geom_point() +  
  labs(title = "Scatter Plot of Height vs Weight",  
        x = "Height (cm)", y = "Weight (kg)") +  
  theme_classic(base_size=18) +  
  theme(plot.title = element_text(hjust=0.5))
```


散布図: 応用 – テーマ全体の変更例 –

theme_classic(コード例)



theme_minimal



演習問題 –使用データ–

使用データ: anorexia_df (MedDataSets パッケージ)

データ概要:

- 拒食症患者を対象としたランダム化比較試験 (RCT) のデータ
- 被験者数: 72名
- 3群 (標準ケア、認知行動療法、家族療法) の割り付け
- 各患者の介入前後の体重変化を記録
- 変数
 - Prewt: 介入前体重 (kg)
 - Postwt: 介入後体重 (kg)
 - Treat: グループ
Cont: 対照群 (標準ケア) / CBT: 認知行動療法群 / FT: 家族療法群

演習問題 –図の作成–

介入前体重と介入後体重の関係を表す散布図を治療群ごとに点の色、形状を変えて作成してください。

1. MedDataSets からデータセット `anorexia_df` の読み込み
2. `ggplot()` でx軸, y軸に対応する変数とグループ変数を指定
3. `geom_point()` で 散布図に
4. `lab()` でタイトル、軸ラベルを追加
5. (余裕があれば) `theme()`関数などで文字サイズ等の体裁を調整

Rコード例 -過食症データの散布図作成-

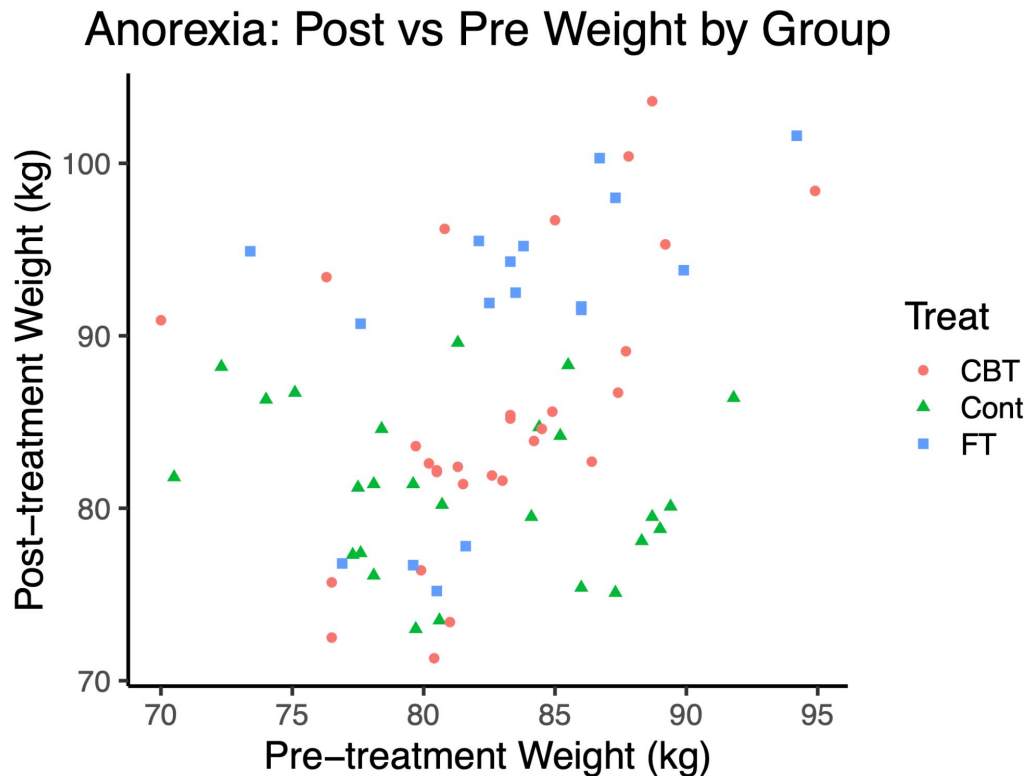
```
# 1. データの読み込み、確認
install.packages("MedDataSets") # 初回のみ
library(MedDataSets)
data("anorexia_df")
head(anorexia_df)
```

```
> head(anorexia_df)
  Treat Prewt Postwt
1  Cont  80.7   80.2
2  Cont  89.4   80.1
3  Cont  91.8   86.4
4  Cont  74.0   86.3
5  Cont  78.1   76.1
6  Cont  88.3   78.1
```

Rコード例 -過食症データの散布図作成-

```
# 2. ggplotで描画
ggplot(anorexia_df, aes(x = Prewt, y = Postwt, color =
                        Treat, shape = Treat)) +
  geom_point(size = 2) +
  labs(
    title = "Anorexia: Post vs Pre Weight by Group",
    x = "Pre-treatment Weight (kg)",
    y = "Post-treatment Weight (kg)") +
  theme_classic(base_size=18) +
  theme(plot.title = element_text(hjust=0.5))
```

出力例 -過食症データの散布図作成-



Chap 3. 生存時間データの可視化

- 生存時間データの概要：打ち切り, at risk, 記述統計
- カプランマイヤー曲線：単群, 群ごと

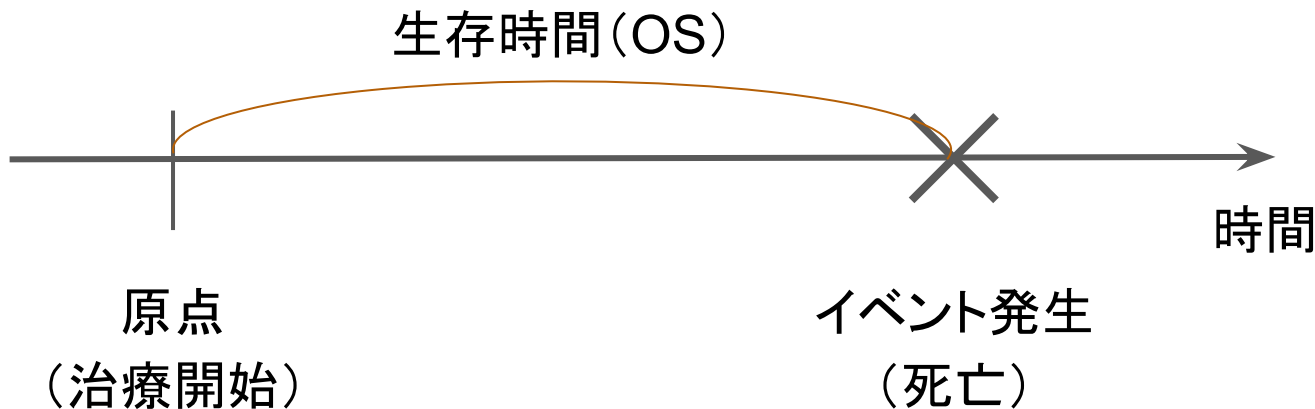
生存時間データ

生存時間データ(Time-to-event data)とは？

- ある時点からある事象(イベント)が起こるまでの時間

例: 治療開始から死亡まで(全生存期間, overall survival: OS)

手術から再発まで(無増悪生存期間, progression free survival: PFS)

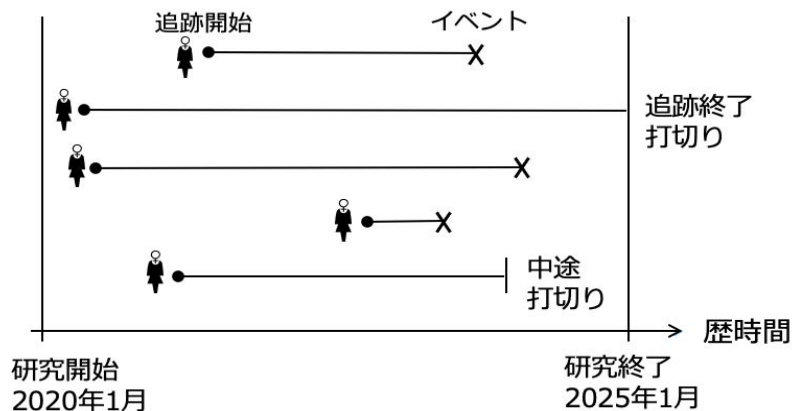


打ち切り(Censoring)

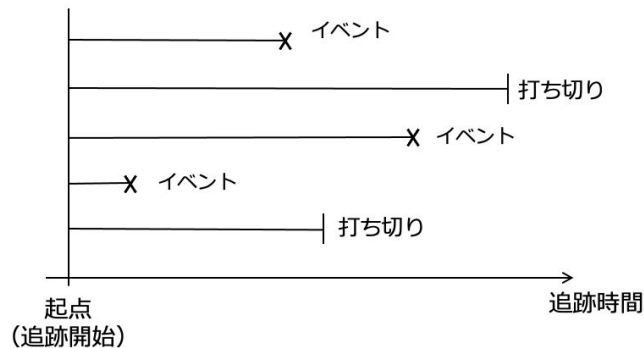
生存時間データには打ち切り(Censoring)が存在

- 追跡期間中にイベントが起きない(追跡終了打ち切り)
- 追跡ができない場合(研究途中での脱落、中途打ち切り)

例: コホート研究における打ち切り(医療データ科学第3回, p.12, 13)



起点を揃える



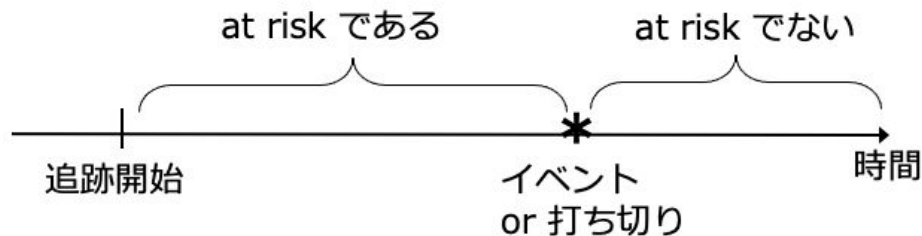
at risk

打ち切りが起きると

- ある時点からイベントが発生するまでの時間は観測不可能
- 打ち切り時点まではイベントが起きていないという情報をもっている

→ at risk の考慮

at risk: ある時点で、追跡の対象となっており、かつ、
興味のあるイベントを起こしていない



生存時間データの記述統計

カプラン・マイヤー (Kaplan-Meier, KM) 法

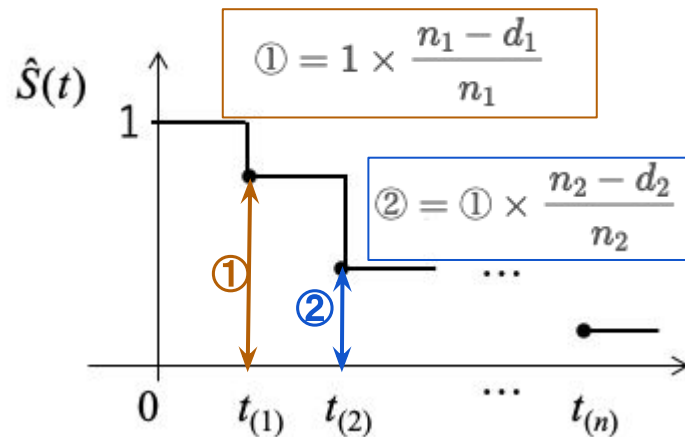
- 生存関数 $S(t)$ を at risk を考慮した下記の方法で推定
生存関数 $S(t)$: 時点 t までにイベントを起こしていない人の割合に相当

$$\begin{aligned}\hat{S}(t) &= \prod_{j:t_{(j)} < t} \frac{n_j - d_j}{n_j} \\ &= \left(\frac{n_1 - d_1}{n_1} \right) \times \left(\frac{n_2 - d_2}{n_2} \right) \times \cdots \times \left(\frac{n_j - d_j}{n_j} \right)\end{aligned}$$

$t_{(1)}, t_{(2)}, \dots, t_{(n)}$: イベント発生時刻を並べたもの

n_j : $t_{(j)}$ 直前の at risk 集団のサイズ

d_j : $t_{(j)}$ でのイベント発生数



分母を number of at risk で計算

カプラン・マイヤー曲線

データとして以下の2つの変数の組として扱う

- Time(時間変数): イベント発生または、打ち切りまでの時間
- Status(打ち切り変数): その時点でイベントが起きたかどうか
一般的にイベント発生=1、打ち切り=0とすることが多い

Rでのカプラン・マイヤー曲線の描画の流れ:

1. survival パッケージの関数 `survfit()` により生存関数を推定
2. survminerパッケージの関数 `ggsurvplot()` により描画

```
install.packages("survival") # 初回のみ  
library(survival)  
install.packages("survminer")  
library(survminer)
```

survfit() 関数

- 基本構文

```
survfit(Surv(time, event) ~ 1, type = "kaplan-meier", data=df)
```

↑
Surv()関数

二つの変数の組を

df\$time: 時間変数,

df\$event: 打ち切り変数

↑
df: データフレーム

生存時間オブジェクトとして定義

```
# データ (年)
time    <- c(1, 4, 5, 6, 8)
event   <- c(1, 1, 0, 1, 0) # 1=イベント発生, 0=打ち切り
df <- data.frame(time, event)

fit <- survfit(Surv(time, event) ~ 1, type = "kaplan-meier",
data=df)
summary(fit)    # 推定結果のサマリー
```

カプラン・マイヤー曲線 -コード例-

```
> summary(fit)    # 推定結果のサマリー
```

```
Call: survfit(formula = Surv(time, event) ~ 1, data = df, type =  
"kaplan-meier")
```

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
1	5	1	0.8	0.179	0.5161	1
4	4	1	0.6	0.219	0.2933	1
6	2	1	0.3	0.239	0.0631	1

>

at risk 数

イベント数

生存時間関数の推定値

ggsurvplot() 関数

- 基本構文

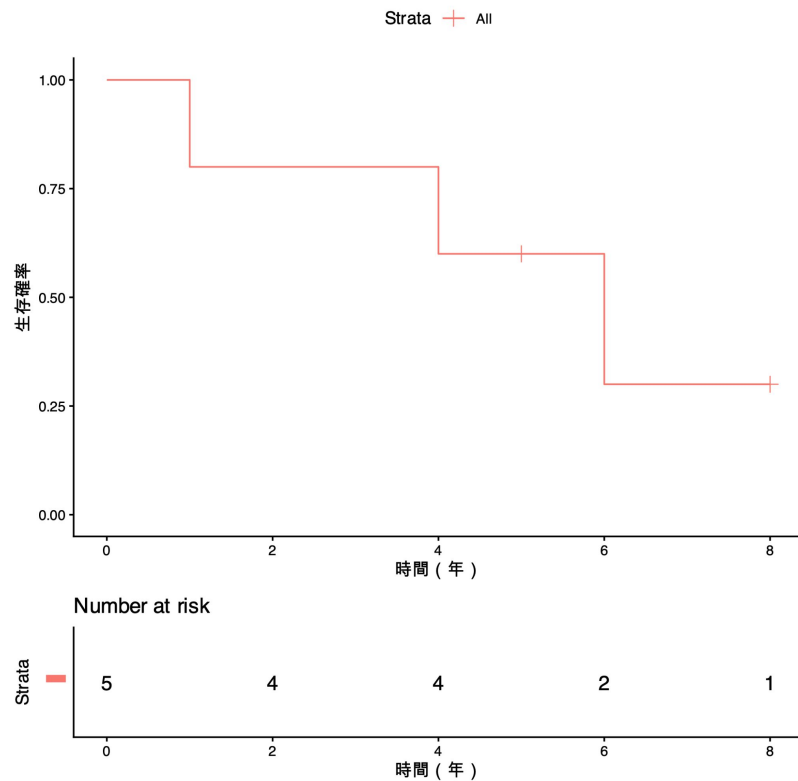
```
ggsurvplot(fit, data = df):
```

最低限必要な引数はsurvfit()の結果とsurvfit()に使用したデータフレーム

以下の詳細な設定が可能

```
ggsurvplot(  
  fit0,  
  data = df,  
  conf.int = FALSE,      # 信頼区間を表示したければ TRUE  
  risk.table = TRUE,      # リスク集合の表を下に表示  
  risk.table.height = 0.25, # 表の高さの比率  
  risk.table.y.text = FALSE,  
  censor.shape = 3,      # 打切りのマークとサイズ  
  censor.size = 3,  
  xlab = "時間 (年)",  
  ylab = "生存確率",  
  ggtheme = theme_classic()  
)
```

Kaplan-Meier 曲線 -出力例-



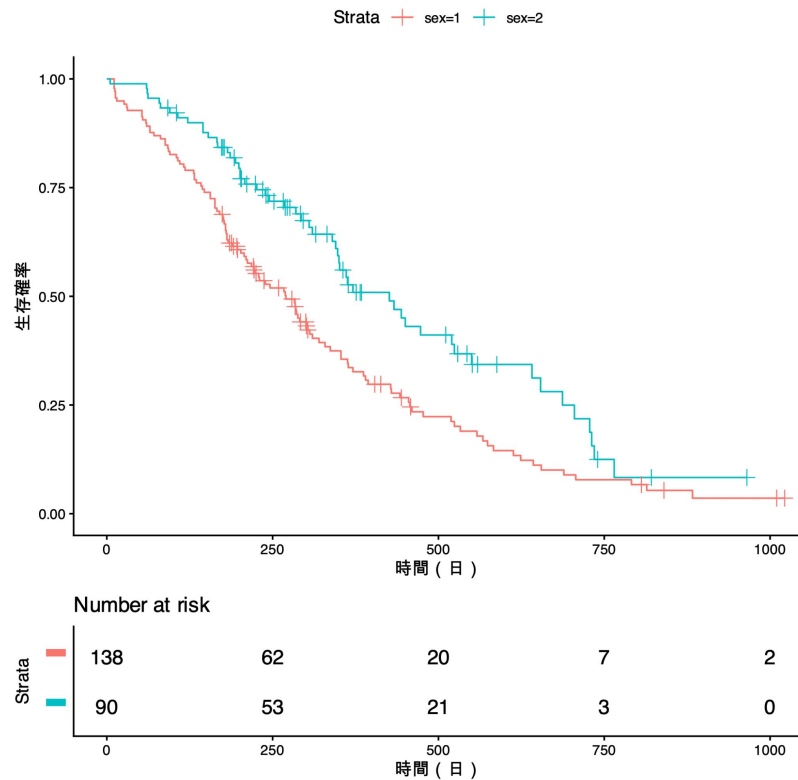
群ごとのKM曲線 -コード例-

```
fit_lung<- survfit(Surv(time, status) ~ sex, data = lung)
ggsurvplot(
  fit_lung,
  data = lung,
  conf.int = FALSE,      # 信頼区間を表示したければ TRUE
  risk.table = TRUE,     # リスク集合の表を下に表示
  risk.table.height = 0.25, # 表の高さの比率
  risk.table.y.text = FALSE, # 表の縦軸のラベルの表示有無
  censor.shape = 3,      # 打切りのマークとサイズ
  censor.size = 3,
  xlab = "時間 (日)",
  ylab = "生存確率",
  ggtheme = theme_classic()
)
```

分けたい変数を
~ の右に指定



群ごとのKM曲線 -出力例-



提出課題

本日、第7回の実習で実行したRscriptファイルを提出してください

提出方法: KULMSの課題提出機能

提出期限: 6/3(水) 23:55

予告:コーディングテスト 6/9(火)13:20-

これまでの実習の習得度を評価するためにコーディングテストを実施します

- 提示された問題をRで実装・処理し, いつもの課題提出と同じ形式で出力を提出する形式
- 実施時間は30分(13:20-13:50)
- 範囲: 第6回までの内容(グラフ作成は含まない)
- 持ち込みは可, web検索および生成AIの利用は不可

Appendix 1. Base Rを用いた グラフ作成

→ 参考資料

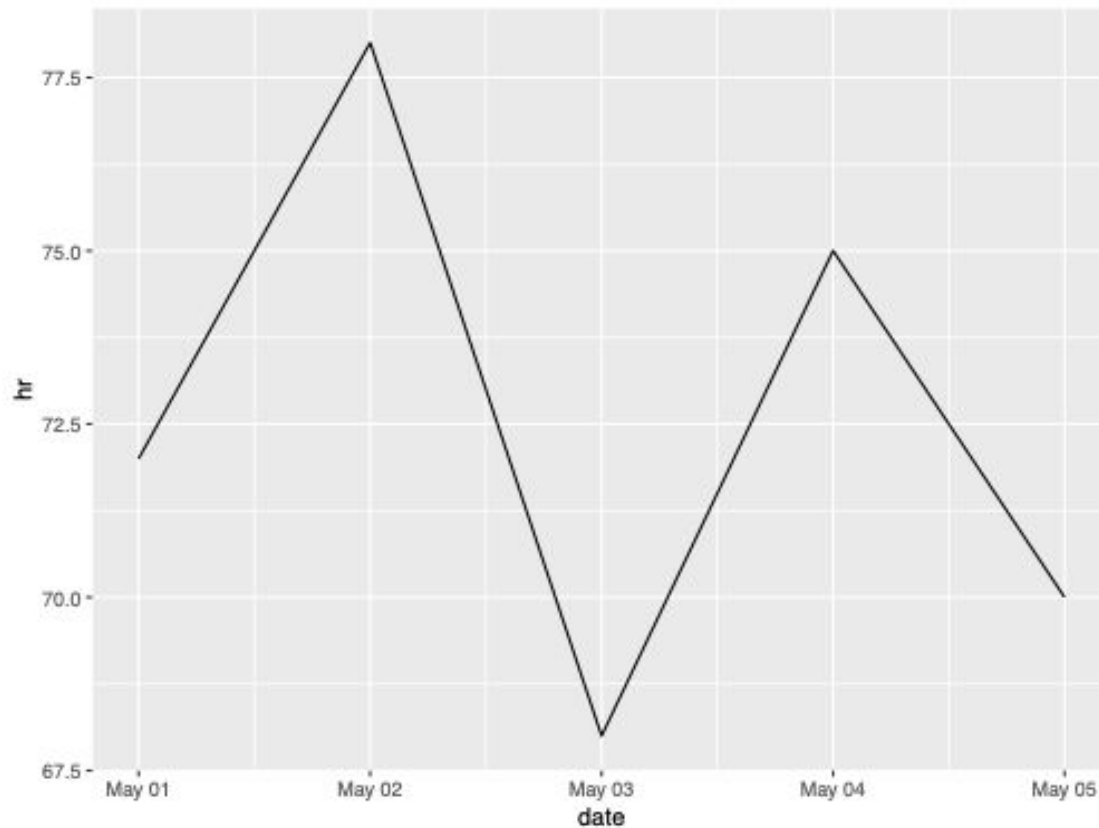
Appendix 2. 時系列プロット

時系列プロット: データの時間的変化の視覚化

折れ線グラフはggplot() + geom_line() で描画

```
dates <- as.Date(c("2025-05-01", "2025-05-02", "2025-05-03",  
"2025-05-04", "2025-05-05"))  
heart_rate <- c(72, 78, 68, 75, 70)  
data_ts <- data.frame(date = dates, hr = heart_rate)  
  
# 時系列プロット  
ggplot(data_ts, aes(x = date, y = hr)) +  
  geom_line()
```

時系列プロット: データの時間的変化の視覚化



plot()関数の引数typeとの対応

plot(type = "...") の各指定は、ggplot2 では以下に対応します。

- ・"p"(点): geom_point()
- ・"l"(線): geom_line()
- ・"b", "o"(点と線): geom_line() と geom_point() の併用

時系列プロット: 線の種類の指定

線種の指定には `geom_line()` の引数 `linetype` を使用

```
ggplot(data_ts, aes(x = date, y = hr)) +  
  geom_line(linetype = "dashed")
```

`linetype` の指定と線の種類の対応

0	blank	
1	solid	
2	dashed	
3	dotted	
4	dotdash	
5	longdash	
6	twodash	

時系列プロット: 2群の時系列プロット

複数グループの設定には `aes()` の引数 `group` を使用

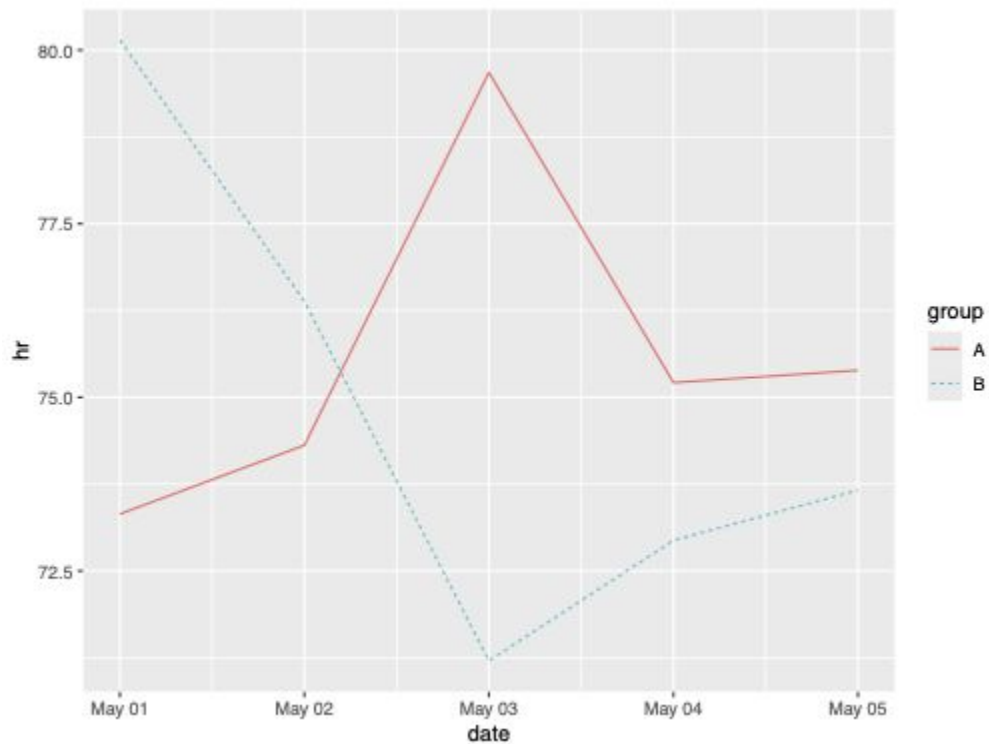
- ・引数 `group` はどの点のデータを繋げるかを設定している
 - ・色だけ変えてしまうと、違うグループの線が繋がって描画されてしまう
- ・線の種類は `aes` の引数の `linetype` で指定

```
ggplot(data, aes(x, y, group = "グループ変数名")) +  
  geom_line()
```

時系列プロット: 2群の時系列プロットーコード例ー

```
# サンプルデータ作成
set.seed(123)
data_ts <- data.frame(date = c(dates, dates),
  hr = rnorm(10, mean = 75, sd = 3),
  group = rep(c("A", "B"), each = 5))
# ggplot による重ね書き
ggplot(data_ts, aes(x = date, y = hr,
  group = group, linetype = group, color = group)) +
  geom_line()
```

時系列プロット: 2群の時系列プロットー出力例ー



演習問題

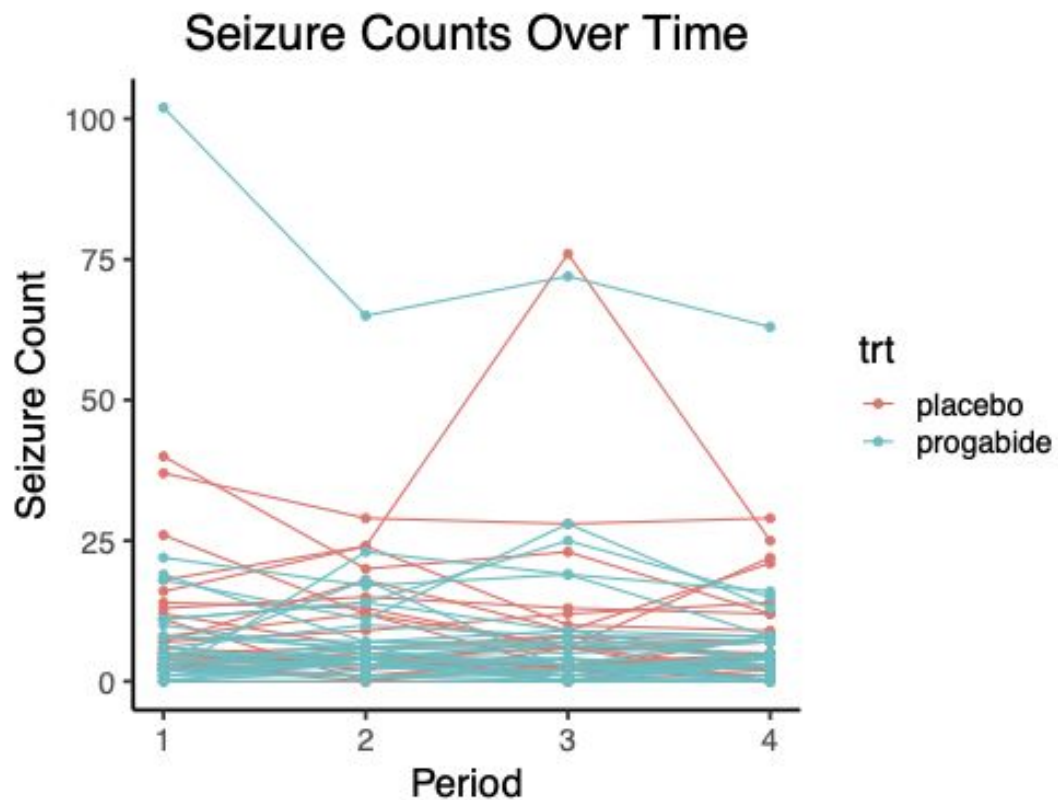
- 使用データ: `epil` (MASS パッケージ)
 - 59人のてんかん患者を対象とした臨床試験データ
 - 各被験者について、介入後2週間ごとの発作回数(計4時点)を記録
 - 介入はプログアバイド (`progabide`) とプラセボ
 - 変数
 - `subject`: 被験者ID
 - `period`: 観察時期(1~4)
 - `y`: 発作回数
 - `trt`: 治療群 (`placebo` / `progabide`)
- 全ての被験者のデータの時系列プロットを描いてみましょう
 - 測定されたデータ点も図示すること(`geom_point()`)
 - 線の色を治療群ごとに変えること(`group`, `color`に別々の変数を設定)
 - タイトルとラベルを適切に設定すること(`lab()`)

Rコード例

```
install.packages("MASS") # 初回のみ
library(MASS)
library(ggplot2)
data("epil")

ggplot(epil, aes(x = period, y = y,
                  group = subject, color = trt)) +
  geom_line() + geom_point() +
  labs(title = "Seizure Counts Over Time",
       x = "Period", y = "Seizure Count") +
  theme_classic(base_size=18) +
  theme(plot.title = element_text(hjust=0.5))
```

出力例



(発展)凡例タイトルの設定

- ・凡例のタイトルはcolorで指定した列名に設定される
- ・`lab(color="凡例テキスト")`と指定することで手動で設定可能
- ・データフレームの列名自体を変えるのも別の解決策

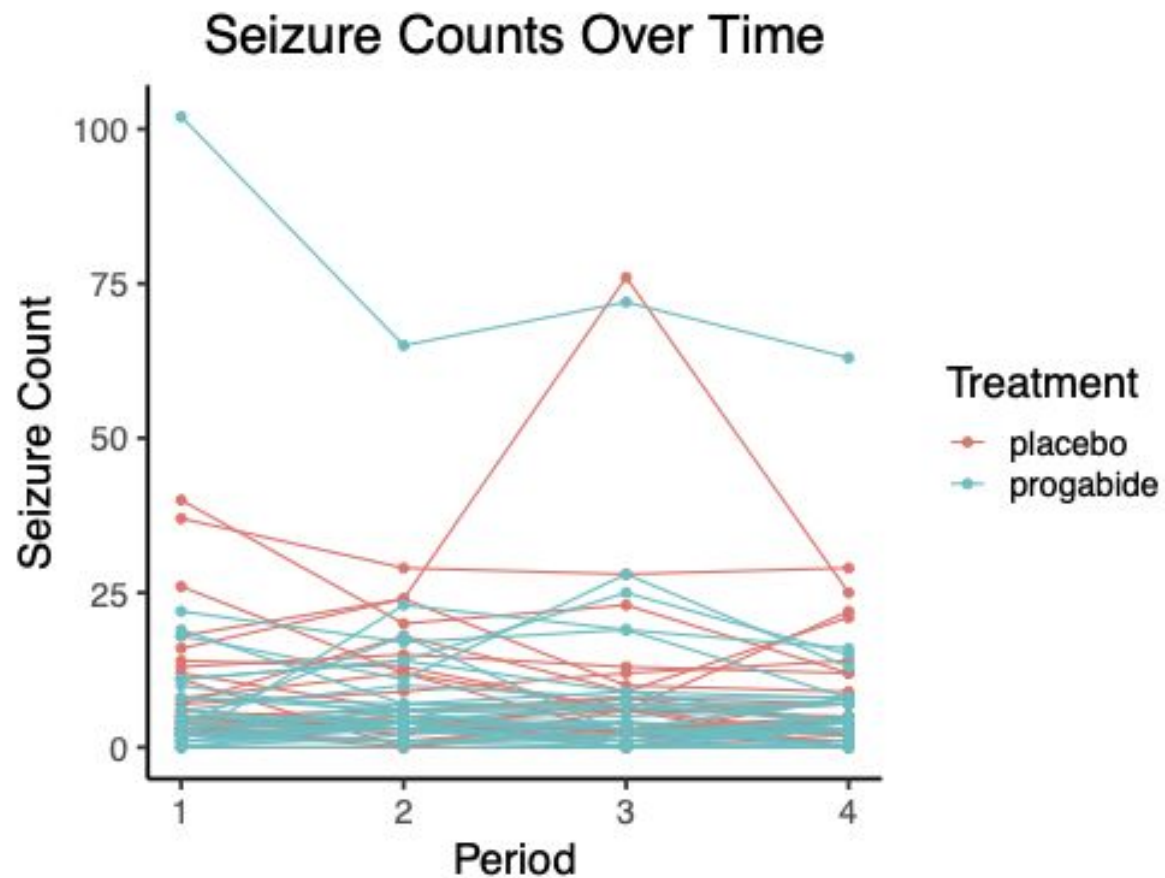
Rコード例

```
install.packages("MASS") # 初回のみ
library(MASS)
library(ggplot2)
data("epil")

ggplot(epil, aes(x = period, y = y,
                  group = subject, color = trt)) +
  geom_line() + geom_point() +
  labs(title = "Seizure Counts Over Time",
       x = "Period", y = "Seizure Count",
       color = "Treatment", linetype = "Treatment") +
  theme_classic(base_size=18) +
  theme(plot.title = element_text(hjust=0.5))
```

* lab() の引数 color, linetype は凡例のタイトルの指定

出力例



Appendix 3. ggplot2を用いた グラフの詳細設定

(参考) 色の手動調整

`scale_color_manual()` により色の設定を変えることが可能
また、透明度は `geom_point()` の引数 `alpha` で制御できる

- `aes(color = グループ変数)` と併用
- `values` に名前付きベクトルで色を対応させる
- カラーコード("#RRGGBB")や色名、`rgb()` も使用可能
- `alpha = 数値 (0~1)` で透明度を指定
- `scale_fill_manual()`, `scale_shape_manual()`, `scale_linetype_manual()` も参照

```
ggplot(anorexia_df, aes(x = Prewt, y = Postwt, color = Treat)) +  
  geom_point(size = 3, alpha = 0.5) +  
  scale_color_manual(values = c(  
    "Cont" = "#E41A1C",  # 赤  
    "CBT"  = "#377EB8",  # 青  
    "FT"   = "#4DAF4A"   # 緑))
```

(参考) 軸の手動調整

`scale_x_continuous()` や `scale_y_continuous()` により軸の目盛やラベルを設定

- `breaks`: 目盛位置を指定 (数値ベクトル)
- `labels`: 目盛ラベルを指定 (文字列ベクトルや関数)
- `limits`: 表示範囲の明示的な指定 (軸の上下限)
- `expand`: 軸の余白 (デフォルトで若干広がる)
- `scale_x_log10()`, `scale_x_date()`, `scale_x_discrete()` など参照

```
ggplot(anorexia_df, aes(x = Prewt, y = Postwt)) +  
  geom_point(pch = 16) +  
  scale_x_continuous(  
    name = "Pre-treatment Weight (kg)",  
    breaks = seq(70, 95, 5),  
    limits = c(68, 97),  
    expand = c(0, 0))
```

(参考) 凡例の手動調整

- 凡例のテキストサイズは `legend.text`、タイトルは `legend.title` で調整
- 凡例の位置は `legend.position` で調整 ("`top`", "`bottom`", "`left`", "`right`" など)

```
ggplot(data2, aes(x = height, y = weight, color = group)) +  
  geom_point() +  
  theme(  
    legend.title = element_text(size = 14),  
    legend.text  = element_text(size = 12),  
    legend.position = "bottom"  
  )
```

(参考) 背景や枠線のカスタマイズ

theme() により、グリッド線や背景の表示も調整可能

- panel.background, plot.background, panel.grid.major など
- element_blank() で非表示にできる

例: 背景を白、グリッド線を非表示

```
ggplot(data, aes(x = height, y = weight)) +  
  geom_point() +  
  theme(  
    panel.background = element_rect(fill = "white"),  
    panel.grid.major = element_blank(),  
    panel.grid.minor = element_blank(),  
    plot.background  = element_rect(fill = "gray95")  
  )
```


(参考) ggsave()による保存

- 現在の出力やggplot で作成したオブジェクトの保存
- ファイル形式(PNG, PDF, SVG など)は拡張子で指定
- 幅・高さ・解像度なども調整可能

```
# 現在のggplot出力を "myplot.png" として保存
ggsave("myplot.png")
# 明示的にオブジェクトを保存
g <- ggplot(data, aes(x = height, y = weight)) +
  geom_point()
ggsave("scatter.pdf", plot = g, width = 6, height = 4)
# 解像度を指定して保存 (PNG形式で高解像度)
ggsave("myplot_highres.png", dpi = 300)
# 従来通りの方法でも可能
pdf("scatter2.pdf", width = 6, height = 4)
  print(g)
dev.off()
```

(参考) 複数の図の表示とラベル付け

複数の ggplot による図を1枚の図として並べて表示するには、ggpubr パッケージの `ggarrange()` 関数が便利

- `labels=` で各図に "A", "B" などのラベルを付けることが可能
- `ncol=` や `nrow=` でレイアウトを指定

patchwork パッケージなども有用

```
ggarrange(p1, p2,  
          labels = c("A", "B"),  
          ncol = 2)
```

(参考) 回帰直線や基準線の追加

geom_abline() や geom_smooth() を用いて直線や回帰直線を追加可能

```
+ geom_abline(intercept = 0, slope = 1, linetype = "dashed")
```

```
+ geom_smooth(method = "lm", se = FALSE, color = "red")
```